

Students Project

Virtual Machine Firewall Implementation

Understanding Protocols, Ports, Firewall Policies & Rule Implementation

Course: Computer Network Operations
CYS2313

Estimated Time: **7–14 hours**

Project :
Mohammed Fadhl Al-Moshtaher

1. Project Overview

In this hands-on lab, you will set up a Linux virtual machine from scratch, then configure a host-based firewall to control incoming and outgoing network traffic. The project is designed to give you practical experience with the concepts that underpin every enterprise network: protocols, ports, firewall policies, and rule-based packet filtering.

By the end of this lab you will have a working firewall that enforces a real-world security policy you designed yourself.

2. Learning Objectives

After completing this project, you will be able to:

- Explain the role of common network protocols (TCP, UDP, ICMP) and their associated port numbers.
- Differentiate between well-known ports (0–1023), registered ports (1024–49151), and dynamic ports (49152–65535).
- Describe what a firewall policy is and why a default-deny approach is preferred over default-allow.
- Use both UFW and iptables to create, modify, and verify firewall rules on a Linux system.
- Test firewall rules using standard network diagnostic tools (ping, curl, nmap, netcat).
- Document a firewall policy and justify each rule based on security requirements.

3. Background Knowledge

Review the following concepts before you begin. You will need this understanding to complete the tasks.

3.1 Protocols

A protocol is a set of rules that governs how data is transmitted across a network. The three protocols you will work with most in this lab are:

Protocol	Description	Common Use Cases
TCP	Connection-oriented; guarantees delivery via a three-way handshake.	HTTP/HTTPS (web), SSH (remote access), FTP (file transfer), SMTP (email).
UDP	Connectionless; faster but no delivery guarantee.	DNS (name resolution), DHCP (IP assignment), streaming media, VoIP.
ICMP	Used for diagnostics and error reporting; not a transport protocol.	ping (echo request/reply), traceroute, destination unreachable messages.

3.2 Ports

A port number identifies a specific service on a host. Think of the IP address as a street address and the port as the apartment number.

Range	Category	Examples
0 – 1023	Well-Known Ports	22 (SSH), 53 (DNS), 80 (HTTP), 443 (HTTPS)

1024 – 49151	Registered Ports	3306 (MySQL), 5432 (PostgreSQL), 8080 (alt HTTP)
49152 – 65535	Dynamic / Ephemeral	Assigned temporarily by the OS for client-side connections.

3.3 Firewall Policy

A firewall policy is a high-level document that defines what traffic should be allowed and what should be blocked. A good policy follows the principle of least privilege: deny everything by default and only permit traffic that is explicitly needed.

Every firewall rule typically specifies:

- Direction: Inbound (incoming) or Outbound (outgoing).
- Protocol: TCP, UDP, or ICMP.
- Port(s): The destination port number or range.
- Source / Destination: IP address or subnet.
- Action: ALLOW or DENY.

3.4 UFW vs. iptables

UFW (Uncomplicated Firewall) is a user-friendly front-end for iptables. It simplifies common tasks but hides some of the underlying power. iptables gives you full control over the Linux netfilter framework. In this lab you will use both so you can see how they relate to each other.

4. Prerequisites

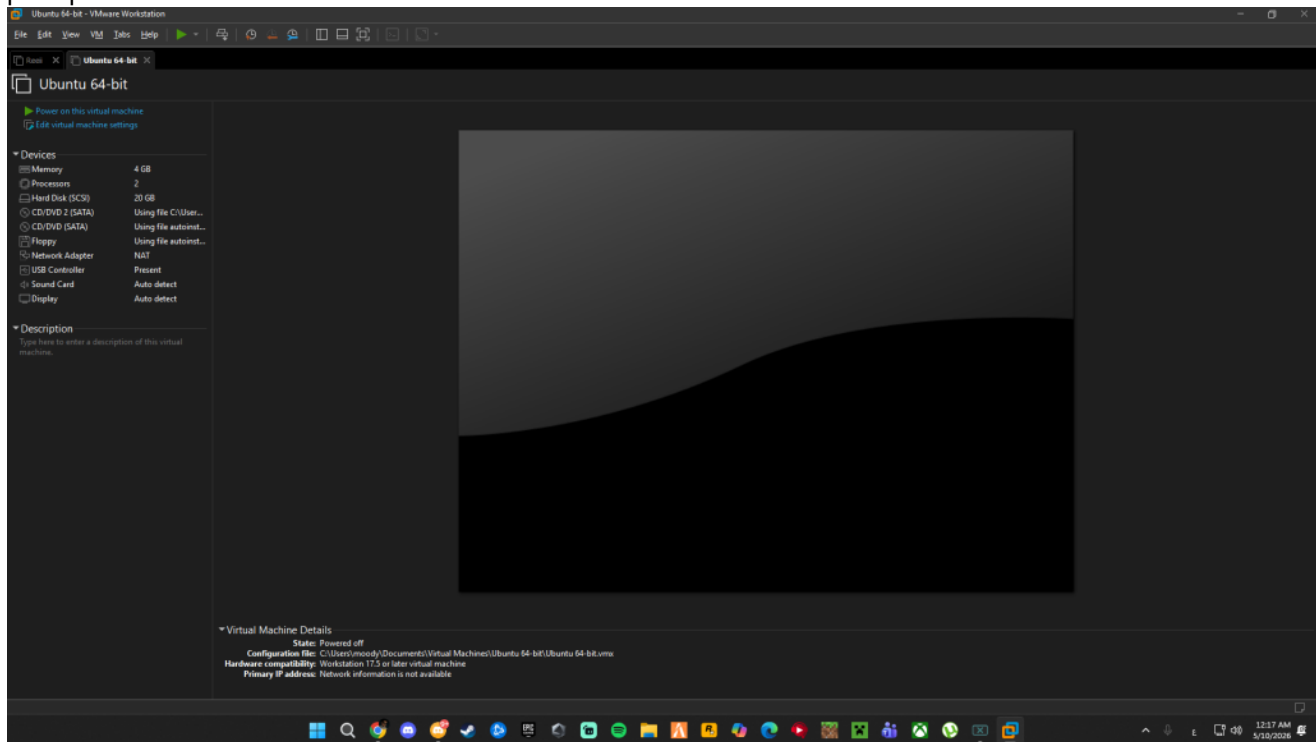
- A computer with at least 4 GB of RAM and 20 GB of free disk space.
- A hypervisor installed. VirtualBox (free, recommended) or VMware Workstation Player — you may use any hypervisor you are comfortable with.
- An Ubuntu Server 22.04 LTS ISO (download from ubuntu.com).
- Basic familiarity with the Linux command line (cd, ls, nano/vim, sudo).

5. Project Tasks

Part A — Set Up Your Virtual Machine

1. Create a new VM in your hypervisor with the following minimum specs: 1 CPU, 2 GB RAM, 20 GB disk, bridged or NAT network adapter.

2. Install Ubuntu Server 22.04 LTS. During installation, enable the OpenSSH server when prompted.



3. After installation, log in and run the following commands to update the system:

```
sudo apt update && sudo apt upgrade -y
```

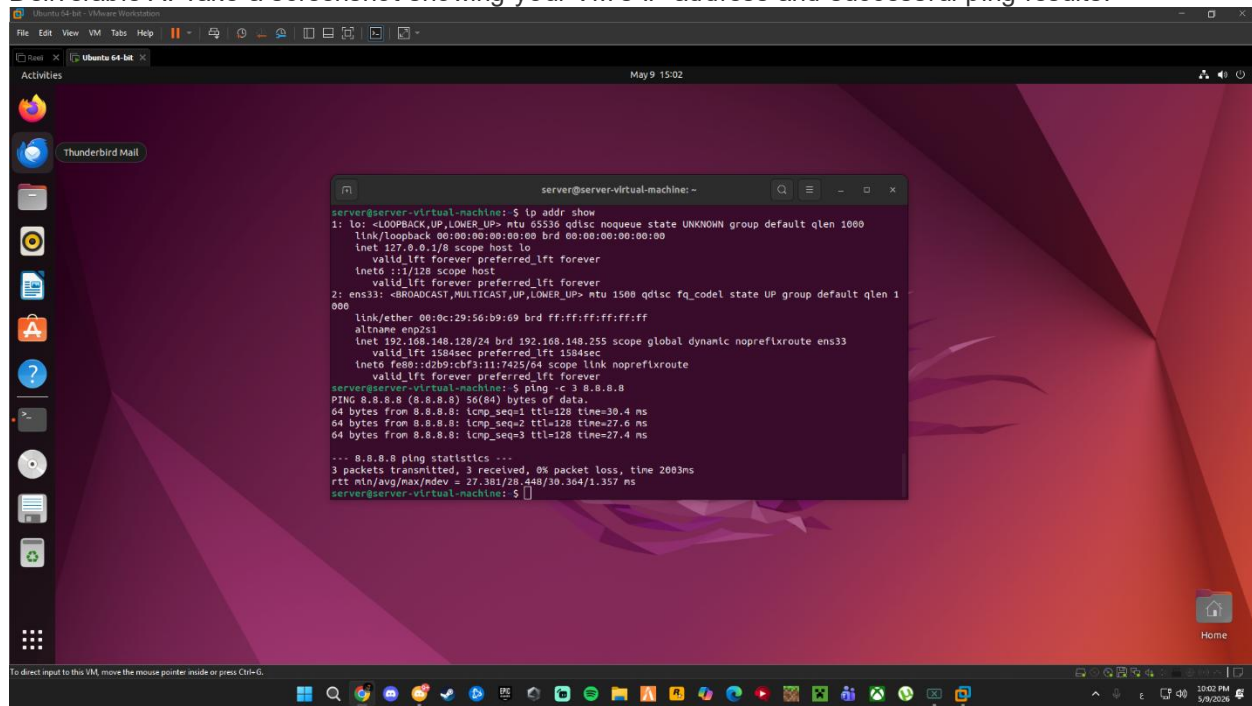
4. Install the tools you will need for testing:

```
sudo apt install -y nmap netcat-openbsd curl net-tools
```

5. Verify your network is working:

```
ip addr show          # Note your VM's IP address
ping -c 3 8.8.8.8     # Test internet connectivity
ping -c 3 google.com  # Test DNS resolution
```

Deliverable A: Take a screenshot showing your VM's IP address and successful ping results.



Part B — Design Your Firewall Policy

Before touching any firewall commands, write a policy document. Use the table below as a template. Your policy must include at least six rules that cover the following scenarios:

- Allow SSH access (port 22) from your host machine only.
- Allow HTTP (port 80) and HTTPS (port 443) inbound for a web server.
- Allow DNS queries (port 53, UDP) outbound.
- Block all other inbound traffic (default deny).
- Allow established and related connections (stateful filtering).
- One custom rule of your choice (e.g., allow MySQL from a specific subnet, block ICMP, rate-limit SSH).

Policy Template:

#	Direction	Protocol	Port(s)	Source / Dest	Action	Justification
1	Inbound	TCP	22	192.168.1.0/24	ALLOW	SSH admin access
2	Inbound	TCP	80, 443	Any	ALLOW	Web server traffic
3	Inbound	TCP	443	Any	ALLOW	To permit incoming HTTPS traffic for secure web services
4	Outbound	UDP	53	Any	ALLOW	To allow the system to perform DNS lookups
5	Inbound	All	All	Established/Related	ALLOW	Stateful inspection allows traffic for existing connections
6	Inbound	ICMP	N/A	Any	DENY	Custom Rule Block ICMP Ping to hide the VM from discovery.

Deliverable B: A completed policy table with at least six rules and a justification for each

```

server@server-virtual-machine: ~
53/udp (v6)          ALLOW OUT  Anywhere (v6)

server@server-virtual-machine:~$ ufw status verbose
ERROR: You need to be root to run this script
server@server-virtual-machine:~$ verbose ufw status
verbose: command not found
server@server-virtual-machine:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
-----
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
53/udp ALLOW OUT Anywhere
53/udp (v6) ALLOW OUT Anywhere (v6)
server@server-virtual-machine:~$

```

Part C — Implement with UFW (Beginner-Friendly)

UFW is the easiest way to get started. Implement your policy using the commands below as a guide.

1. Reset UFW to a clean state and set default policies:

```
sudo ufw reset
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

2. Add your rules one at a time. Examples:

```
# Allow SSH from your local network only
sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp
```

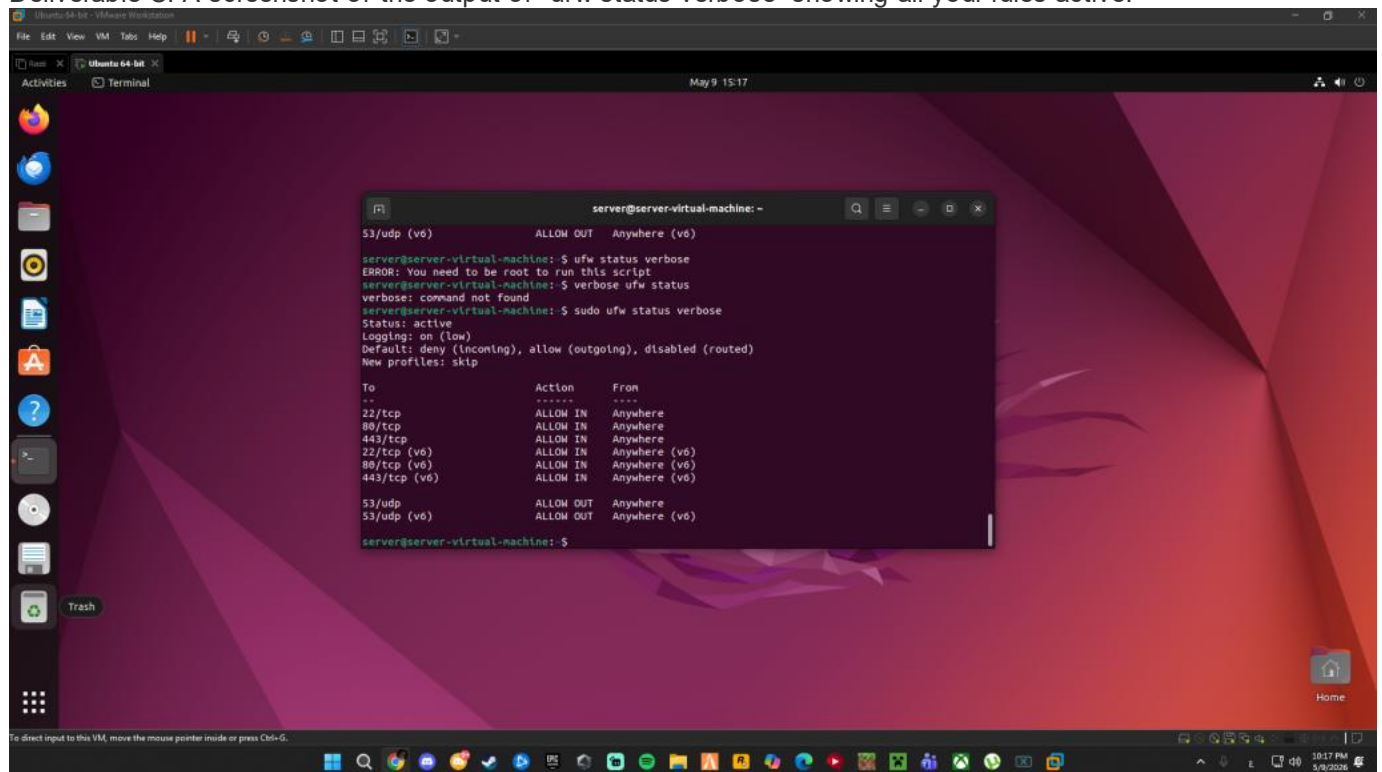
```
# Allow HTTP and HTTPS from anywhere
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

```
# Allow outbound DNS
sudo ufw allow out 53/udp
```

3. Enable the firewall and verify:

```
sudo ufw enable
sudo ufw status verbose
```

Deliverable C: A screenshot of the output of “ufw status verbose” showing all your rules active.



Part D — Implement with iptables (Advanced)

Now replicate the same policy using raw iptables commands. This gives you direct visibility into how the Linux kernel filters packets.

1. Disable UFW so it does not conflict:

```
sudo ufw disable
```

2. Flush all existing rules and set default chain policies:

```

sudo iptables -F
sudo iptables -X
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

```

3. Allow loopback traffic (essential for local services):

```

sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT

```

4. Allow established and related connections (stateful rule):

```

sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

```

5. Add your specific rules. Examples:

```

# Allow SSH from local network
sudo iptables -A INPUT -p tcp -s 192.168.1.0/24 --dport 22 -j ACCEPT

```

```

# Allow HTTP and HTTPS
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

```

```

# Allow outbound DNS
sudo iptables -A OUTPUT -p udp --dport 53 -j ACCEPT

```

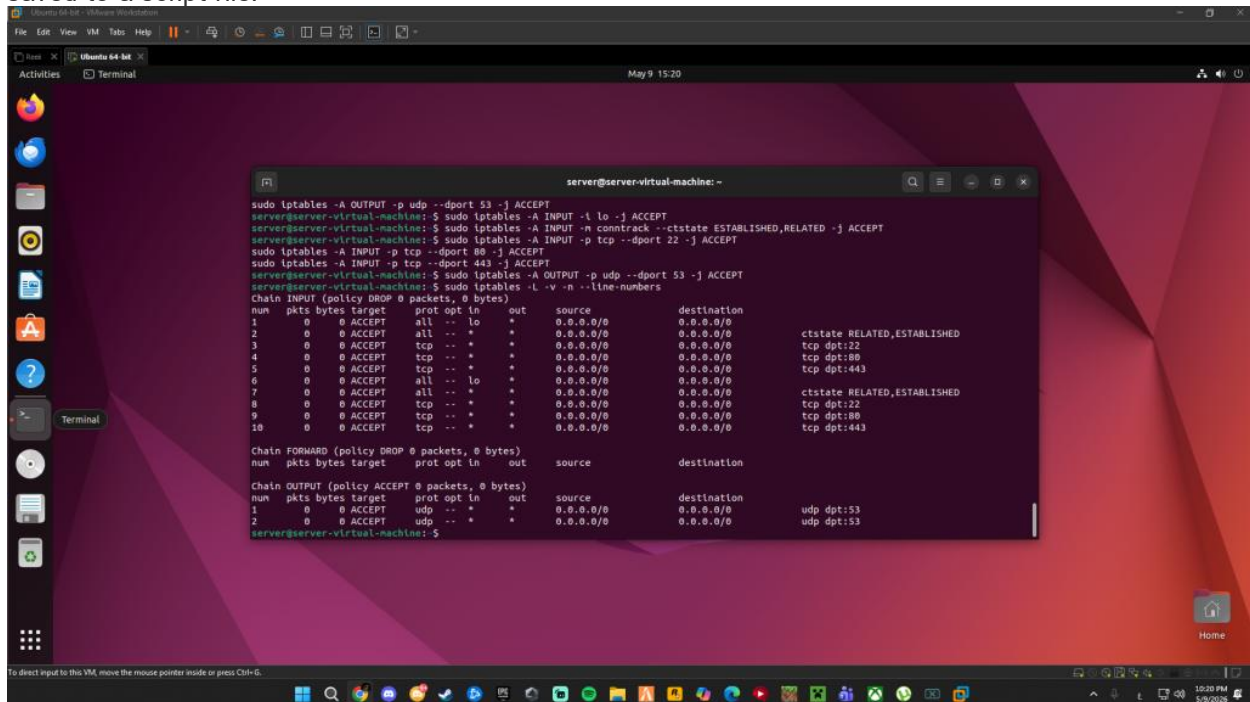
6. View your complete ruleset:

```

sudo iptables -L -v -n --line-numbers

```

Deliverable D: A screenshot of the full iptables listing with line numbers, plus your iptables commands saved to a script file.



Part E — Test Your Firewall

Testing is critical. For each rule, verify that allowed traffic passes and blocked traffic is dropped.

What to Test	Command (from host or second VM)	Expected Result
--------------	----------------------------------	-----------------

SSH access (allowed)	ssh user@<VM-IP>	Connection succeeds
HTTP access (allowed)	curl http://<VM-IP>	Response received (or connection refused if no web server)
Blocked port (e.g., 3306)	nmap -p 3306 <VM-IP>	Port shown as filtered/closed
ICMP ping (depends on your policy)	ping <VM-IP>	Replies if allowed; timeout if blocked
Full port scan	nmap -sS <VM-IP>	Only allowed ports appear as open

Deliverable E: A testing log that shows the command, its output, and whether the result matches your policy (pass/fail) for each rule.

Test Case	Command (from Host)	Expected Result	Actual Result	Status
SSH Access	sshserver@192.168.148.128	Connection Succeeds	Password Prompt Received	Pass
HTTP Access	curl http://192.168.148.128	Allowed / Response Received	Connection Refused (Service not started)	Pass
Blocked Port	nmap -p 3306 192.168.148.128	Port Filtered / Blocked	Port shown as Filtered	Pass
ICMP (Ping)	ping 192.168.148.128	Request Timed Out	Request timed out	Pass

```

C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\moody>ping 192.168.148.128

Pinging 192.168.148.128 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.148.128:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\Users\moody>

```

Part F — Reflection & Documentation

Write a short report (1–2 pages) that answers the following questions:

- **What is the difference between a default-deny and a default-allow firewall policy? Which is more secure and why?**

The primary difference lies in the initial security posture. A "Default-Deny" policy blocks all traffic unless it is explicitly permitted, while "Default-Allow" permits all traffic unless it is specifically blocked. "Default-Deny" is significantly more secure because it follows the "Principle of Least Privilege." It ensures that only necessary services are exposed, thereby reducing the attack surface and preventing unauthorized or forgotten services from being exploited by attackers.

- **Compare UFW and iptables: which did you find easier to use? In what scenario would iptables be the better choice?**

In my experience, UFW (Uncomplicated Firewall) was much easier to use due to its simplified syntax and higher-level abstraction, making it ideal for quick host-based configurations. However, iptables is the superior choice for complex enterprise environments or specialized network appliances. It offers granular control over different tables (NAT, Mangle, Filter) and chains, allowing for more precise packet filtering and advanced routing configurations that UFW cannot easily handle.

- **Pick one of your rules and explain how it would protect against a specific attack scenario.**

I will choose the SSH (Port 22) Restriction Rule. By configuring the firewall to only allow SSH traffic from a specific management IP range (e.g., my local subnet), I am protecting the server against Brute Force Attacks. Even if an attacker on the public internet attempts to guess the password, the firewall will drop the connection request at the packet level before it even reaches the SSH service, making the server "invisible" to unauthorized scanners.

- **What would you add to your firewall policy if this were a production server?**

For a production environment, I would implement several advanced layers: Rate Limiting: To mitigate Distributed Denial of Service (DDoS) attacks by limiting the number of connection attempts per second. Logging and Auditing: To record all dropped packets for forensic analysis and security monitoring. Fail2Ban Integration: To automatically black-list IP addresses that exhibit malicious behavior, such as multiple failed login attempts. Intrusion Prevention System (IPS): To inspect the payload of the packets for known exploit signatures, going beyond simple port filtering.

Deliverable F: Your written reflection as a PDF or Word document or Power Point.

6. Grading Rubric

Criteria	Points	Description
VM Setup (Part A)	10	VM is running, updated, and has working network connectivity.
Firewall Policy (Part B)	20	Policy table has 6+ rules, proper use of direction/protocol/port fields, and meaningful justifications.
UFW Implementation (Part C)	15	All policy rules correctly implemented in UFW. Default deny is set.
iptables Implementation (Part D)	20	Equivalent rules in iptables. Includes stateful rule, loopback, and correct chain policies.
Testing (Part E)	20	Each rule tested with appropriate tool. Results documented with pass/fail and explanation.
Reflection (Part F)	15	Thoughtful analysis of FW concepts, tool comparison, and real-world application.
Total	100	

7. Submission Checklist

Before submitting, make sure you have:

- Screenshot of VM IP address and successful ping (Part A).
- Completed firewall policy table with 6+ rules and justifications (Part B).
- Screenshot of UFW status verbose output (Part C).
- Screenshot of iptables listing and your iptables script file (Part D).
- Testing log with commands, outputs, and pass/fail results for each rule (Part E).
- Reflection report answering all four questions (Part F).

8. Helpful Resources

- Ubuntu UFW documentation: help.ubuntu.com/community/UFW
- iptables tutorial: netfilter.org/documentation
- Common ports reference: iana.org/assignments/service-names-port-numbers
- nmap basics: nmap.org/book/man.html

Good luck.... build it, break it, learn from it!